Design Patterns III

Lukas Holik

lukasholik@cs.aau.dk

(based on slides of Gabriela Montoya, thanks)

Creational Patterns

- **Simple Factory**
- **Factory Method**
- **Abstract Factory**
- **Builder**
- ...

Structural Patterns

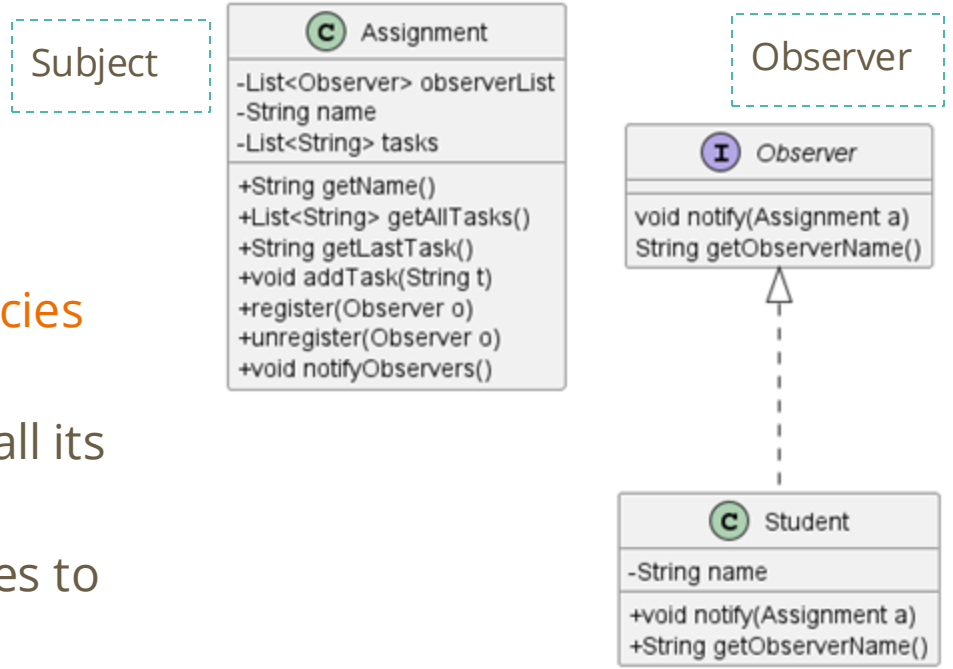
- **Decorator**
- **Adapter**
- **Composite**
- ...

Behavioral Patterns

- **Template Method**
- Observer
- Command
- Strategy
- Visitor
- ...

Observer Pattern

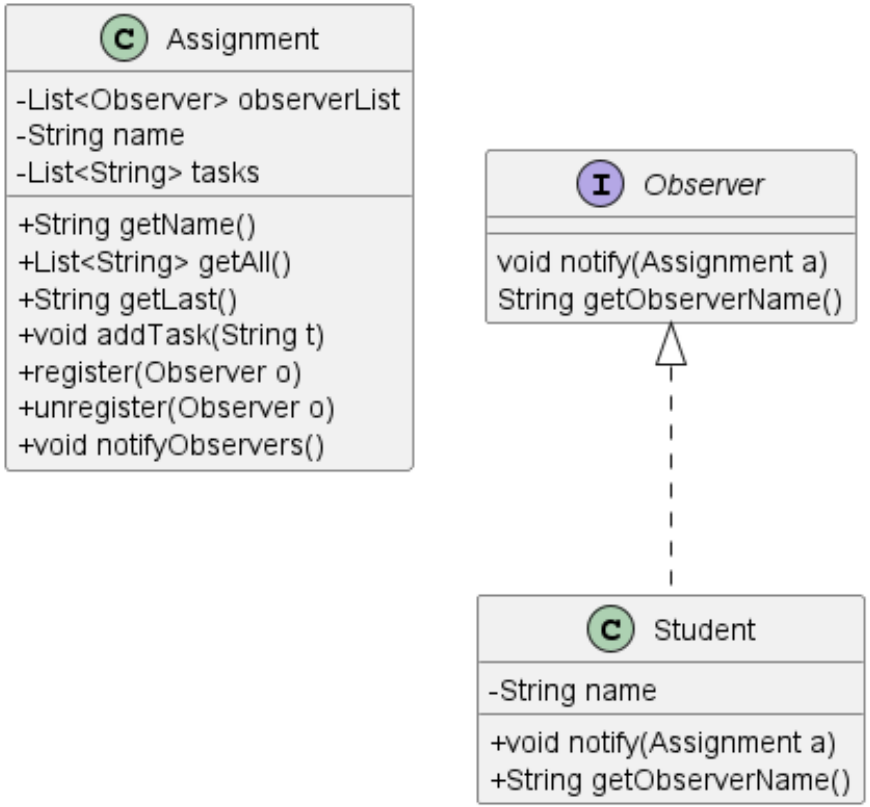
- Defines **one-to-many** dependencies between objects.
- When one object changes state, all its **dependents are notified.**
- Observers can register themselves to get notifications about a subject.
  - They can unregister if they don't want to be notified anymore.
- Subjects and observers make up a **loosely coupled** system.




```
public void addTask(String t) {
    tasks.add(t);
    notifyObservers();
}
public void notifyObservers() {
    for (Observer o : observerList) {
        o.notify(this);
    }
}
```

Select a good asymptotic bound for the method *addTask*

(a) O(1)
(b) O(log n)
(c) O(n)
(d) O(n^2)




```
public void addTask(String t) {
    tasks.add(t);
    notifyObservers();
}
```

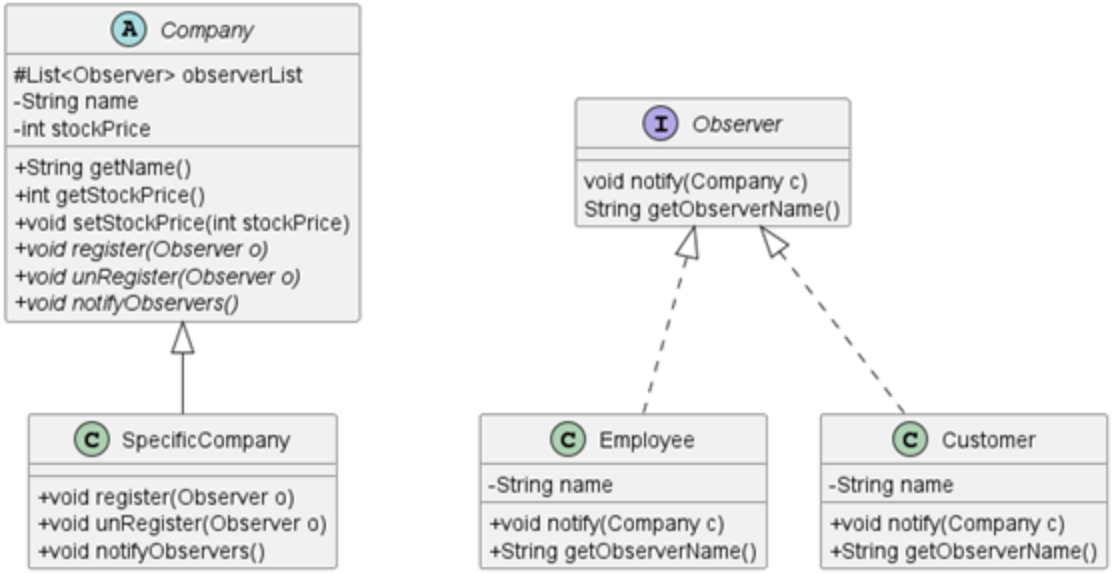
Company Stock Observer

```
public void setStockPrice(int stockPrice) {
    this.stockPrice = stockPrice;
    notifyObservers();
}
```

```
public void register(Observer o) {
    observerList.add(o);
}

public void unRegister(Observer o) {
    observerList.remove(o);
}

public void notifyObservers() {
    for (Observer o : observerList) {
        o.notify(this);
    }
}
```




```
Observer roy = new Employee("Roy");
Observer kevin = new Employee("Kevin");
Company abcLtd = new SpecificCompany("ABC Ltd. ");
abcLtd.register(roy);
abcLtd.register(kevin);
abcLtd.setStockPrice(5);
abcLtd.unRegister(kevin);
abcLtd.setStockPrice(50);
```

Command Pattern

```
public interface Runnable {
    void run();
}
```

- **Requests** are represented as **objects** (command).
- Allows for parameterized clients, queues of requests.
- Supports undoable operations.
- Roles: command, receiver, invoker, client.

receiver

```
class Holder {
    private int value;
    public Holder(int v) {
        value = v;
    }
    ...
    public void setValue(int v) {
        this.value = v;
    }
}
```

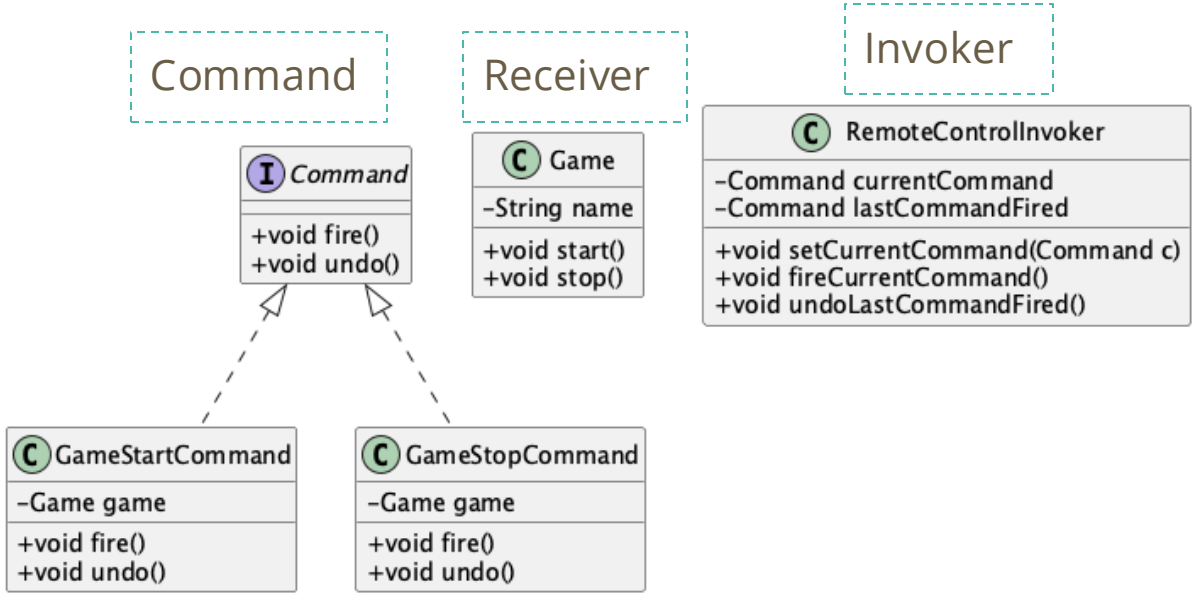
```
public class IncreaseValue implements Runnable
{
    Holder state;
    public IncreaseValue(Holder h) {
        state = h;
    }
    public void run() {
        state.setValue(state.getValue()+1);
    }
}
```

command

Command Pattern for Remote Controller

```
class RemoteControlInvoker {
  ...
  public void fireCurrentCommand() {
    currentCommand.fire();
    lastCommandFired = currentCommand;
  }
}
```

```
class GameStartCommand implements Command {
  ...
  public void fire() {
    game.start();
  }
}
```



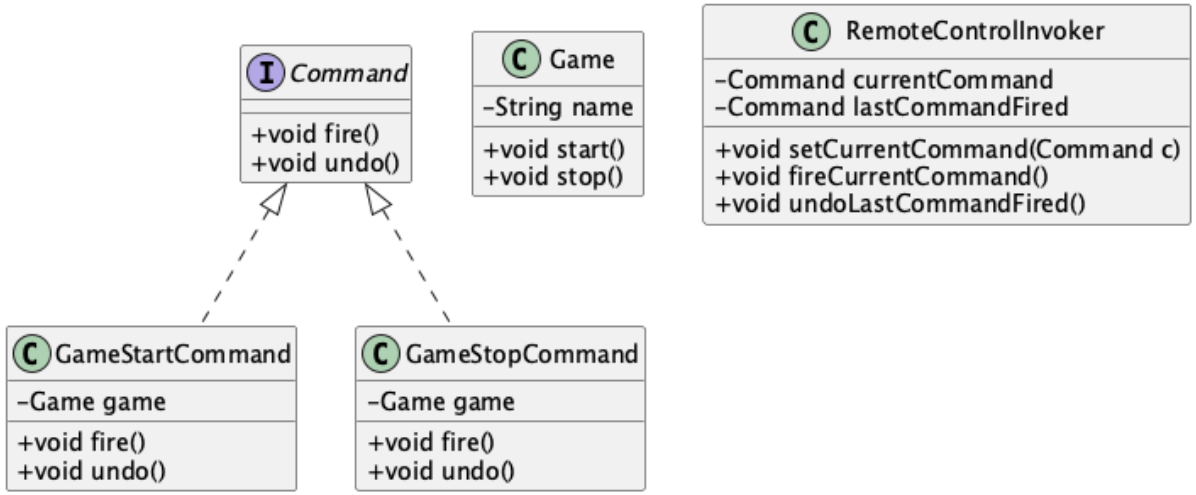

```
Game game = new Game("Golf");
GameStartCommand gameStartCommand = new GameStartCommand(game);
...
invoker.setCurrentCommand(gameStartCommand);
invoker.fireCurrentCommand();
```

Client

Please select zero, one, or more options

Which class needs to handle the checked exceptions that *start* could throw?

(a) Game
(b) GameStopCommand
(c) GameStartCommand
(d) RemoteControlInvoker




```
class RemoteControlInvoker {
  ...
  public void fireSelectedCommand() {
    currentCommand.fire();
    lastCommandFired = currentCommand;
  }
}
```

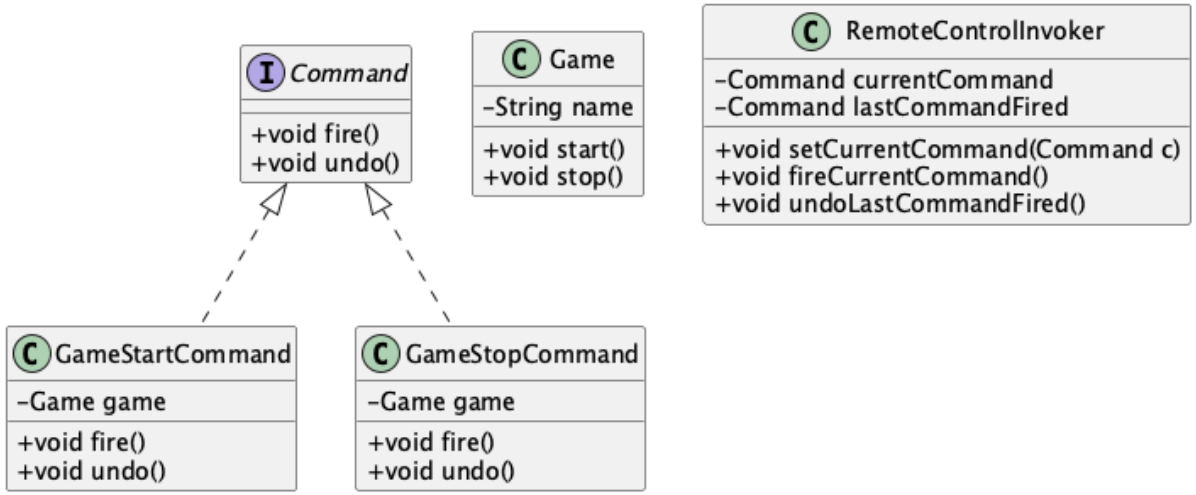
```
class GameStartCommand implements Command {
  ...
  public void fire() {
    game.start();
  }
}
```

Please select zero, one, or more options

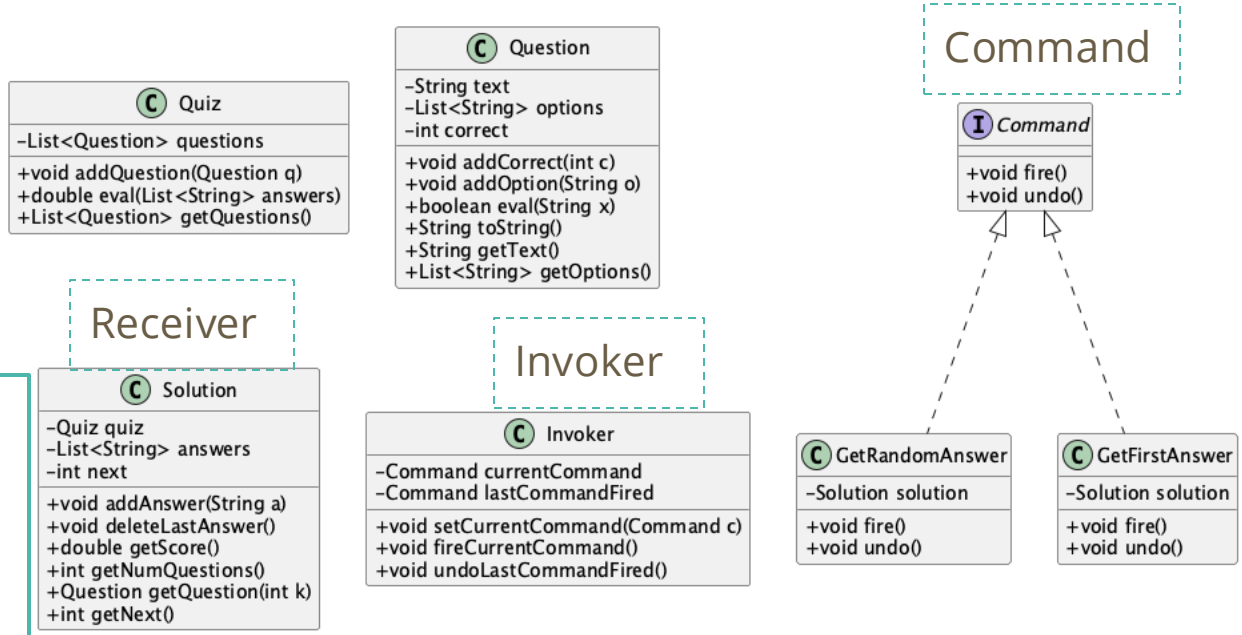
New requirement: we want to be able to undo all executed commands

Which of the following classe needs to be updated?

(a) Game
(b) GameStopCommand
(c) GameStartCommand
(d) RemoteControlInvoker



Command, Quiz Solver




```
class Invoker {
  ...
  public void executeSelectedCommand() {
    currentCommand.fire();
    lastCommandPerformed = currentCommand;
  }
}
```

```
class GetRandomAnswer implements Command {
  ...
  public void fire() {
    int next = solution.getNext();
    if (next < solution.getNumQuestions()) {
      Question q = solution.getQuestion(next);
      List<String> opts = q.getOptions();
      int r = (new Random()).nextInt(opts.size());
      solution.addAnswer(opts.get(r));
    }
  }
}
```

Client

```
Quiz q = createQuiz();
Solution s = new Solution(q);
Invoker i = new Invoker();

Command c = new GetRandomAnswer(s);
i.setCommand(c);
i.fireCurrentCommand();
```

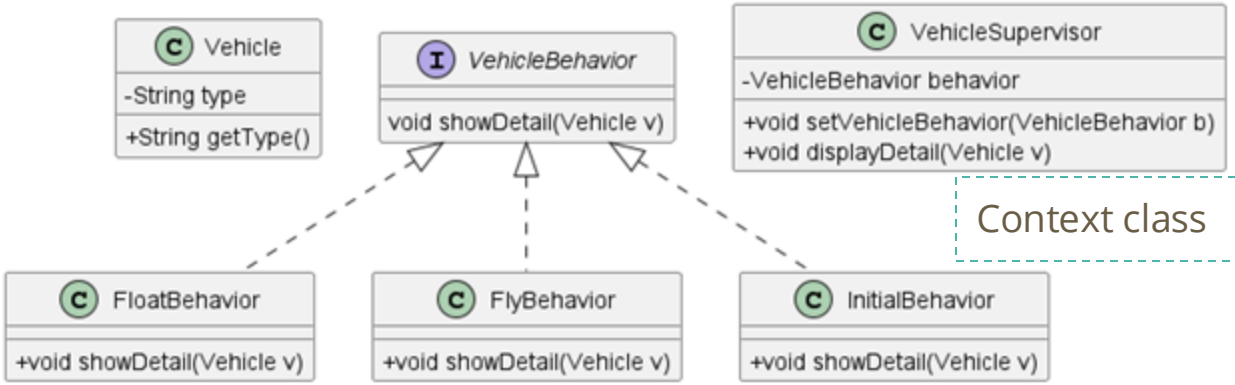
Strategy Pattern

- Defines **a family of algorithms** that can be used interchangeably.
- The **behavior** associated with the object **can be changed** at specific times.
- Example: classes implementing the java.util.Comparator interface provide different sorting criteria.

```
public class TitleComparator implements Comparator<Course> {
    @Override
    public int compare(Course c1, Course c2) {
        String title1 = c1.getTitle();
        String title2 = c2.getTitle();
        return title1.compareTo(title2);
    }
}
```

`TitleComparator` can be passed to `Arrays.sort(T[], Comparator<? super T>)` and `List<E>.sort(Comparator<? super E>)`

Strategy Pattern, Vehicles

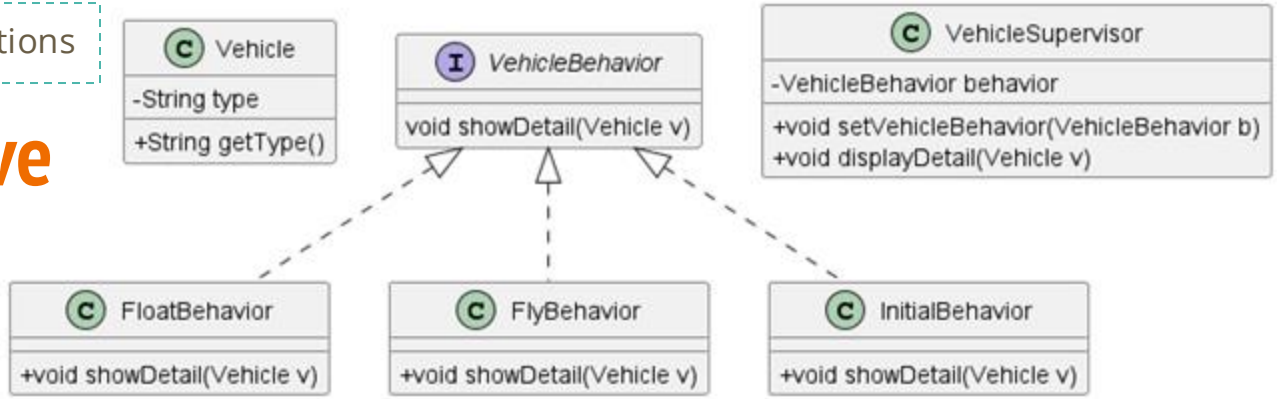



```
class VehicleSupervisor {
    ...
    public void displayDetail(Vehicle vehicle) {
        behavior.showDetail(vehicle);
    }
}
```

```
Vehicle vehicle = new Vehicle("airplane");
VehicleSupervisor supervisor=new VehicleSupervisor(new InitialBehavior());
supervisor.displayDetail(vehicle);
supervisor.setVehicleBehavior(new FlyBehavior());
supervisor.displayDetail(vehicle);
```

Please select zero, one, or more options

New behavior: drive

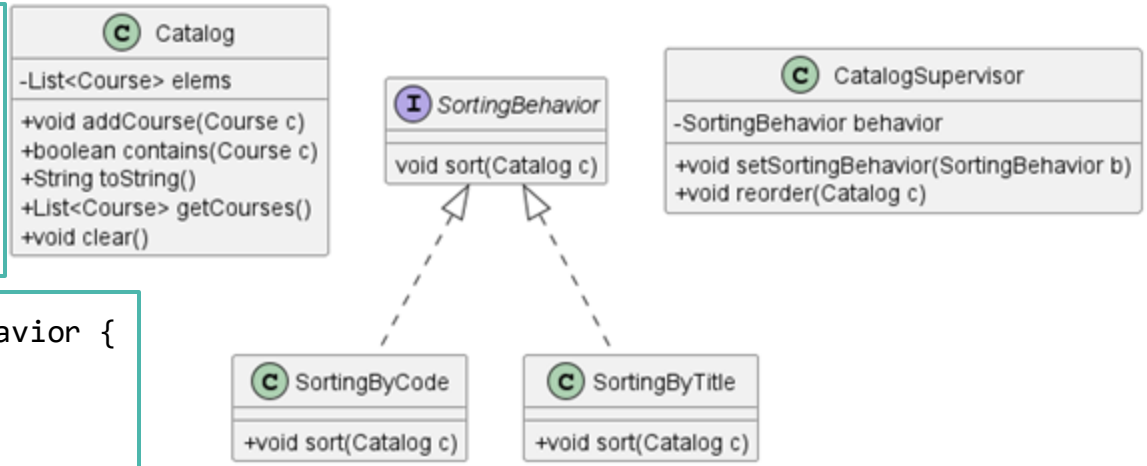


Which of the classes need to be updated?

(a) Vehicle
(b) FloatBehavior
(c) FlyBehavior
(d) InitialBehavior
(e) VehicleSupervisor

Catalogs can be sorted by code or title

```
class CatalogSupervisor {
    ...
    public void reorder(Catalog c) {
        behavior.sort(c);
    }
}
```



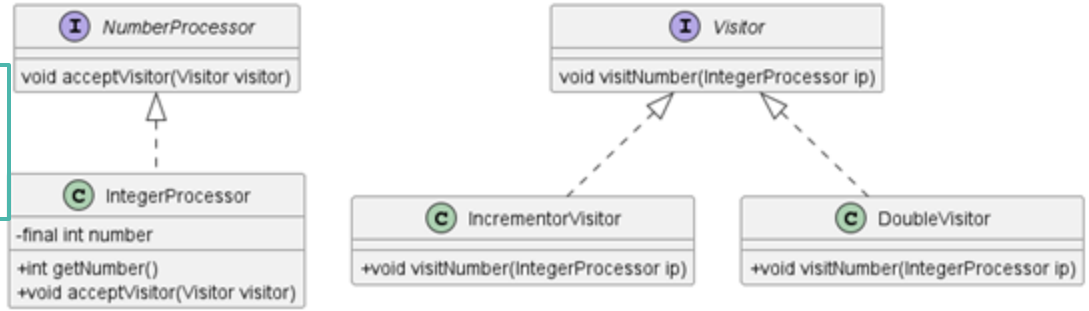

```
class SortingByTitle implements SortingBehavior {
    public void sort(Catalog c) {
        List<Course> es = c.getCourses();
        es.sort(new TitleComparator());
        c.clear();
        for (Course e : es) {
            c.addCourse(e);
        }
    }
}
```

```
Catalog c = createCatalog();
CatalogSupervisor s = new CatalogSupervisor(
                                    new SortingByCode());
s.reorder(c);
System.out.println(c);
s.setSortingBehavior(new SortingByTitle());
s.reorder(c);
System.out.println(c);
```

Visitor Pattern

- Represents an operation to be performed on the elements of an object structure.
- Allows to add new functionality without changing existing code.

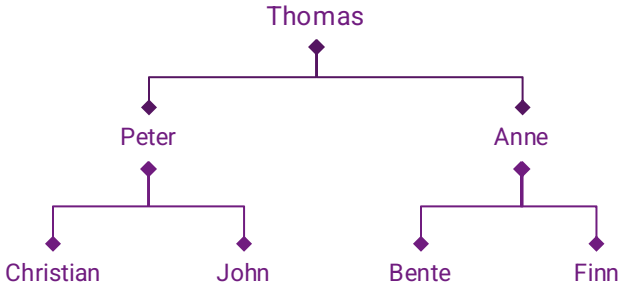
```
NumberProcessor np = new IntegerProcessor();
Visitor visitor = new DoubleVisitor();
np.acceptVisitor(visitor);
```

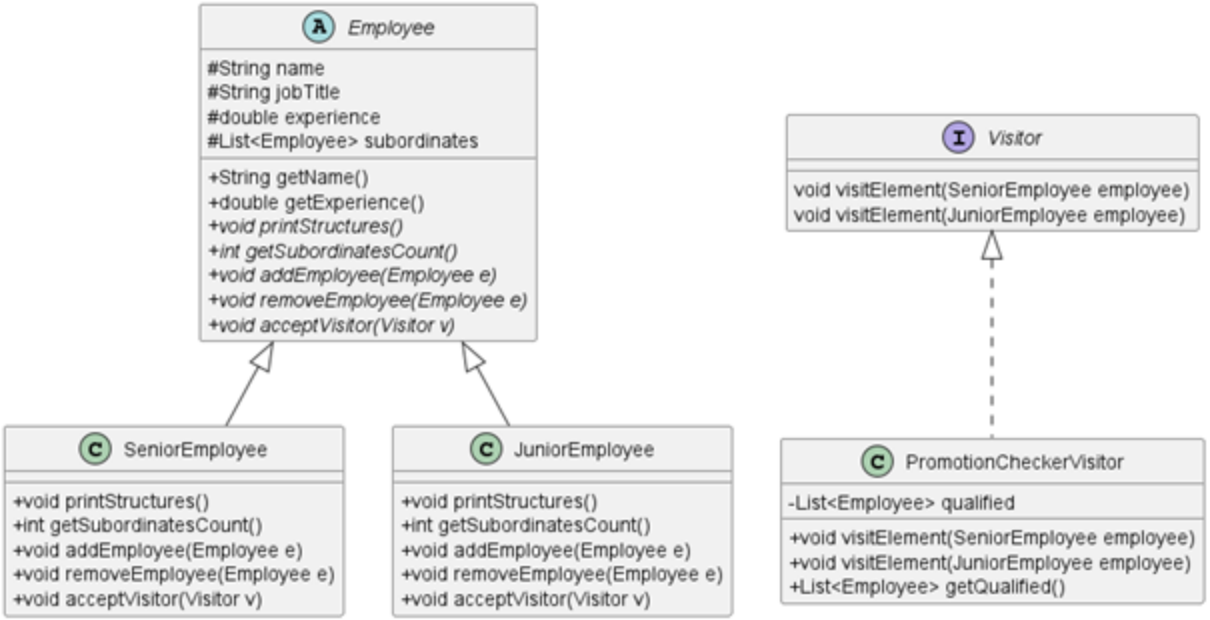



```
class IntegerProcessor {
  ...
  public void acceptVisitor(Visitor visitor){
    visitor.visitNumber(this);
  }
}
```

```
class DoubleVisitor implements Visitor {
  public void visitNumber(IntegerProcessor ip) {
    int temp=ip.getNumber()*2;
    System.out.println("The new value is:"+ temp);
  }
}
```

Visitor for Employees






```
Employee dean = loadFacultyData();
Visitor visitor = new PromotionCheckerVisitor();
dean.acceptVisitor(visitor);
```

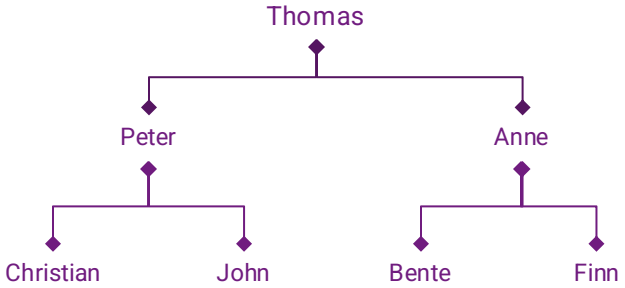
```
public void acceptVisitor(Visitor visitor) {
  visitor.visitElement(this);
}
```

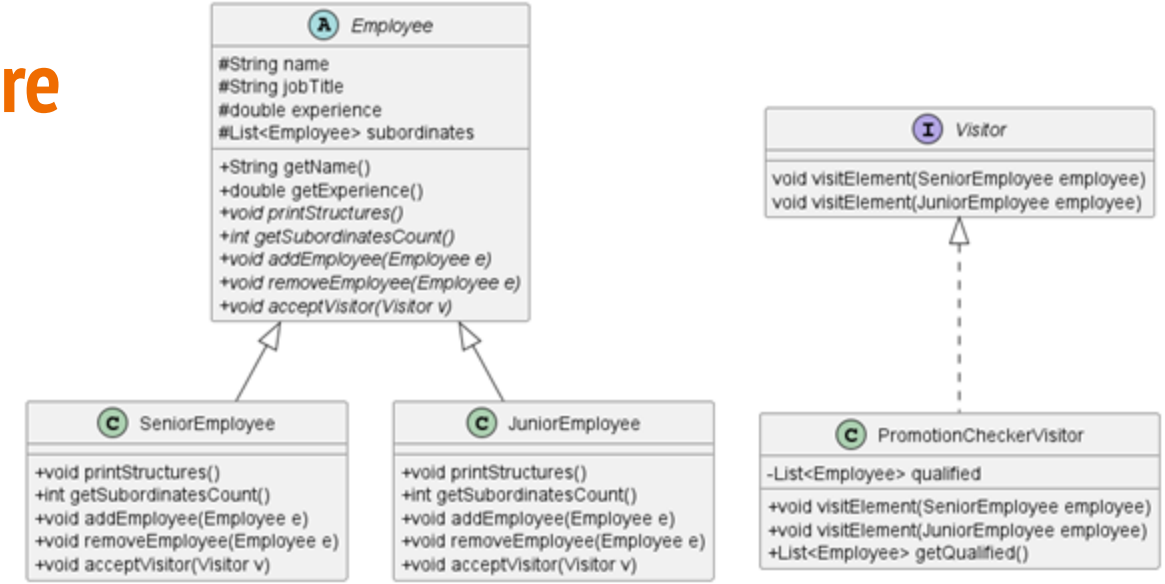
*Employee

```
public void visitElement(SeniorEmployee e) {
  if (e.getExperience()> 15) {
    qualified.add(e);
  }
}
public void visitElement(JuniorEmployee e) {
  if (e.getExperience()> 5) {
    qualified.add(e);
  }
}
```

PromotionCheckerVisitor

How many employees are visited?






```
Employee dean = loadFacultyData();
Visitor visitor = new PromotionCheckerVisitor();
dean.acceptVisitor(visitor);
```

(a) 0
(b) 1
(c) 3
(d) 7

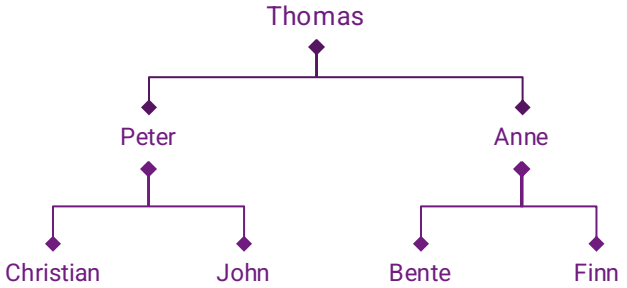
```
public void acceptVisitor(Visitor visitor) {
  visitor.visitElement(this);
}
```

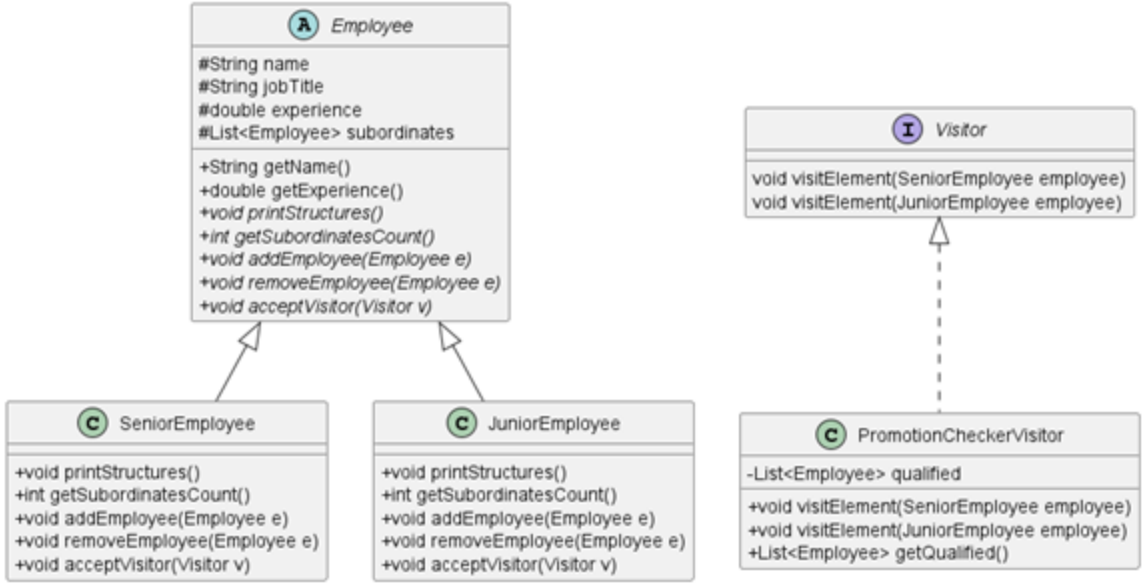
*Employee

```
public void visitElement(SeniorEmployee e) {
  if (e.getExperience()> 15) {
    qualified.add(e);
  }
}
public void visitElement(JuniorEmployee e) {
  if (e.getExperience()> 5) {
    qualified.add(e);
  }
}
```

PromotionCheckerVisitor

Collect the employees using getEmployees






```
Employee dean = loadFacultyData();
Visitor visitor = new PromotionCheckerVisitor();
List<Employee> employees = getEmployees(dean);
for (Employee e : employees)
    e.acceptVisitor(visitor);
}
```

How well does this option work?

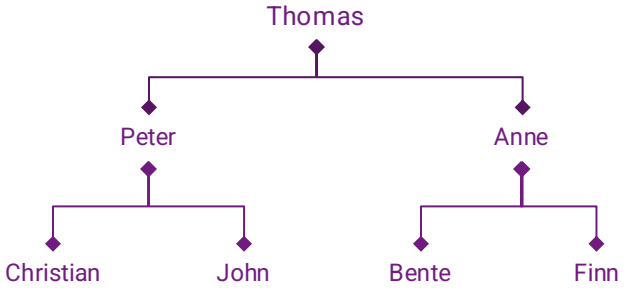
```
public void acceptVisitor(Visitor visitor) {
  visitor.visitElement(this);
}
```

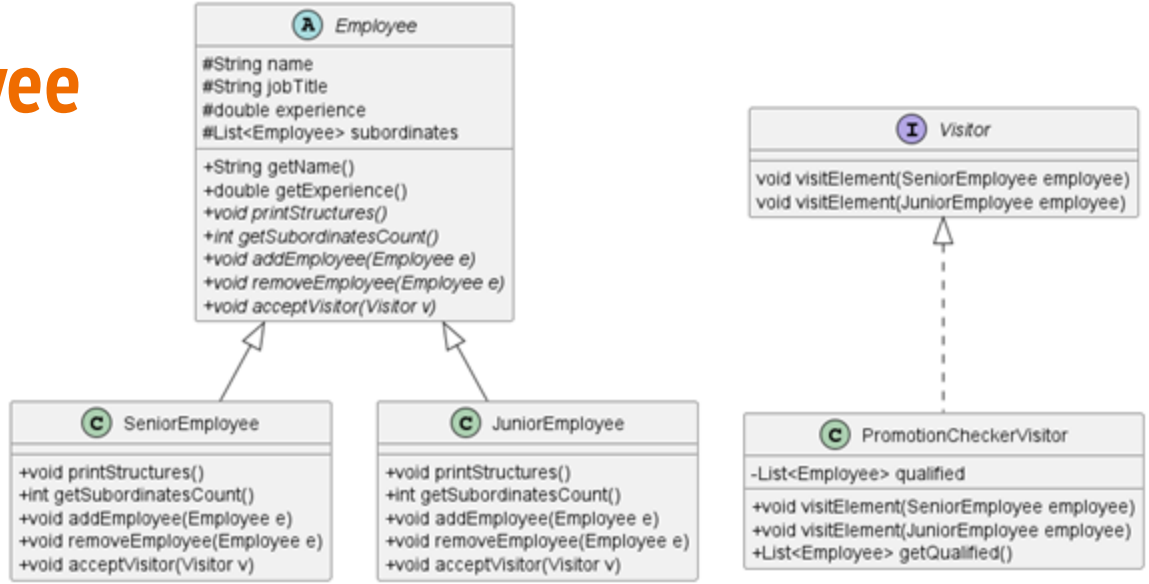
*Employee

```
public void visitElement(SeniorEmployee e) {
  if (e.getExperience()> 15) {
    qualified.add(e);
  }
}
public void visitElement(JuniorEmployee e) {
  if (e.getExperience()> 5) {
    qualified.add(e);
  }
}
```

PromotionCheckerVisitor

Change in SeniorEmployee






```
Employee dean = loadFacultyData();
Visitor visitor = new PromotionCheckerVisitor();
dean.acceptVisitor(visitor);
```

```
public void acceptVisitor(Visitor visitor) {
  visitor.visitElement(this);
  for (Employee s : subordinates) {
    s.acceptVisitor(visitor);
  }
}
```

SeniorEmployee

```
public void acceptVisitor(Visitor visitor) {
  visitor.visitElement(this);
}
```

JuniorEmployee

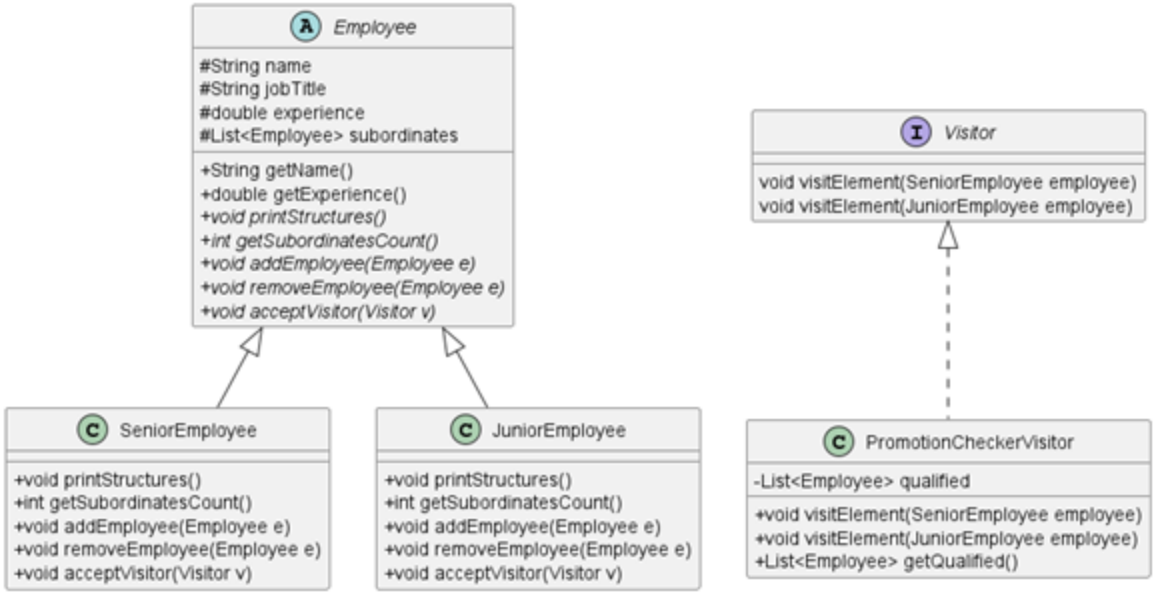
```
public void visitElement(SeniorEmployee e) {
  if (e.getExperience()> 15) {
    qualified.add(e);
  }
}
public void visitElement(JuniorEmployee e) {
  if (e.getExperience()> 5) {
    qualified.add(e);
  }
}
```

PromotionCheckerVisitor

New type of employee: PartTimeEmployee

Which of the classes need to be updated?

(a) Employee
(b) SeniorEmpoyee
(c) JuniorEmployee
(d) PromotionCheckerVisitor



Obtain the number of courses in a catalog

```
public interface Visitor {
    void visit(Catalog c);
}
```

```
class Catalog {
    private Course c;
    private Catalog predecessors, successors;
    public Catalog(Course c) {
        this.c = c;
    }
    ...
    public void accept(Visitor v) {
        v.visit(this);
        if (predecessors != null) {
            predecessors.accept(v);
        }
        if (successors != null) {
            successors.accept(v);
        }
    }
}
```

```
class CountVisitor implements Visitor {
    private int count = 0;
    public void visit(Catalog c) {
        count++;
    }
    public int getCount() {
        return count;
    }
}
```

```
Catalog c = createCatalog();
Visitor v = new CountVisitor();
c.accept(v);
int numCourses = v.getCount();
```

Creational Patterns

- **Simple Factory**
- **Factory Method**
- **Abstract Factory**
- **Builder**
- ...

We studied only some design patterns, there are many more to choose from ...

Structural Patterns

- **Decorator**
- **Adapter**
- **Composite**
- ...

Behavioral Patterns

- **Template Method**
- **Observer**
- **Command**
- **Strategy**
- **Visitor**
- ...

The gang of four, GoF

1. Factory
2. Singleton
3. Observer
4. Strategy

Singleton

```
public class Logger {
    private static Logger instance;
    private level int = 0;

    private Logger() {
        // Private constructor to prevent instantiation
    }

    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String message) {
        if (this.level>0) System.out.println("Log: " + message);
    }

    public setLevel(int l) {this.level = l;}
}
```

- Ensures that the class has only one instance.
- E.g., database connection or configuration manager, log manager.
  Implemented as singletons to ensure only one instance is used throughout the app.